Less is More

Homework 5

1. (a) Both figures plot the training loss of models of different hidden layer width vs the log of the learning rate. Larger width lead to better performance when using the muP scaling.

1.(b): 1

1.(c): ① $\|h_l\|_2 = \Theta(\sqrt{n_l})$ and $\|\Delta h_l\|_2 = \Theta(\sqrt{n_l})$ ensures that $\|h_l\|_{RSM} = \Theta(1)$ and $\|\Delta h_l\|_{RSM} = \Theta(1)$ which prevents the features and their updates from exploding or vanishing.

② $\|h_l\|_{RSM} = \frac{\|h_l\|_{RSM}}{\sqrt{n_l}} = \Theta(1)$, $\|\Delta h_l\|_{RSM} = \frac{\|\Delta h_l\|_{RSM}}{\sqrt{n_l}} = \Theta(1)$

③ $\because h_l = W_l \cdot h_{l-1}$

$\therefore \|h_l\|_2 = \|W_l h_{l-1}\|_2 \le \|W_l\|_2 \|h_{l-1}\|_2 = \Theta(\sqrt{n_l})$

$\therefore \|h_l\|_2 \le O(n_l)$

$\because \Delta h_l + h_l = (\Delta W_l + W_l)(\Delta h_{l-1} + h_{l-1})$

$\therefore \Delta h_l = \Delta W_l \Delta h_{l-1} + \Delta W_l \cdot h_{l-1} + W_l \cdot \Delta h_{l-1} + W_l \cdot h_{l-1} - h_l$

$= \Delta W_l \Delta h_{l-1} + \Delta W_l \cdot h_{l-1} + W_l \cdot \Delta h_{l-1}$

Similarly: $\|\Delta h_l\|_2 \le \|\Delta W_l\|_2 \|\Delta h_{l-1}\|_2 + \|\Delta W_l\|_2 \|h_{l-1}\|_2 + \|W_l\|_2 \|\Delta h_{l-1}\|_2$

$\therefore \|\Delta h_l\|_2 \le \Theta\left(\sqrt{\frac{n_l}{n_{l-1}}}\right) \cdot \|\Delta h_{l-1}\|_2$

I will use mathematical induction to prove the rest:

Base case: Since h_0 will not be changed, therefore $\|\Delta h_0\| = 0$, which stands for $\|\Delta h_0\| = O(\sqrt{n_0})$

Inductive Hypothesis: Assume that for layer $l-1$, $\|\Delta h_{l-1}\| = O(\sqrt{n_{l-1}})$

Inductive steps: $\because \|\Delta h_l\|_2 \le \Theta\left(\sqrt{\frac{n_l}{n_{l-1}}}\right) \cdot \|\Delta h_{l-1}\|_2$

$\therefore \|\Delta h_l\|_2 \le \Theta\left(\sqrt{\frac{n_l}{n_{l-1}}}\right) \cdot O(\sqrt{n_{l-1}})$

$\therefore \|\Delta h_l\|_2 \le \Theta(\sqrt{n_l})$

④ $\|W_l h_{l-1}\| = \Theta(\|W_l\| \cdot \|h_{l-1}\|)$

$\|\Delta W_l h_{l-1}\| = \|\Delta W_l\| \cdot \|h_{l-1}\|$

Less is More

3.(a): Since FP8 has limited numerical precision. the small weights produced by Xavier initialization may go beyond the precision that FP8 can handle. So, $N(0,1)$ may work better.

3.(b): Xavier : $N(0, \frac{1}{\sqrt{d_{in}}})$

$\therefore C = \frac{1}{\sqrt{d_{in}}}$

3.(c): $\Delta y = C \cdot \Delta W \cdot X$

$$\|\Delta y\|_{RMS} = C \cdot \|\Delta W \cdot X\|_{RMS} \leq C \cdot \|\Delta W\|_{RMS-RMS}^{d_{out} \times d_{in}} \cdot \|X\|_{RMS}$$

$$= C \cdot \frac{\|\Delta W \cdot m\|_{RMS}}{\|m\|_{RMS}} \cdot \|X\|_{RMS}$$

$$= C \cdot \sqrt{\frac{d_{in}}{d_{out}}} \cdot \|\Delta W\|_2 \cdot \|X\|_{RMS}$$

$\because \|X\|_{RMS} = 1, \quad C = \frac{1}{\sqrt{d_{in}}}$

$\therefore \|\Delta y\|_{RMS} \leq \frac{1}{\sqrt{d_{in}}} \cdot \sqrt{\frac{d_{in}}{d_{out}}} \cdot \|\Delta W\|_2 \cdot 1 = \frac{1}{\sqrt{d_{out}}} \cdot \|\Delta W\|_2$

$\therefore \|\Delta W\|_2 \leq \sqrt{d_{out}}$

3.(d) $y = C \cdot W \cdot x$ (d_{out})

$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot C \cdot X^T$

$\because$ minibatch $= 1$

$\therefore X^T \in d_{in} \times 1$

$$\begin{pmatrix} y_1' \\ y_2' \\ \vdots \\ y_{out}' \end{pmatrix} \begin{bmatrix} x_1 & x_2 & \cdots & x_{in} \end{bmatrix} = \begin{pmatrix} y_1'x_1 & y_1'x_2 & \cdots & y_1'x_{out} \\ \vdots & & & \vdots \\ y_{out}'x_1 & y_{out}'x_2 & \cdots & y_{out}'x_{out} \end{pmatrix}$$

$\therefore \text{rank}(\text{sign}\,\Delta W L) = 1$

$\therefore \|\text{sign}\,\Delta W L\|_2 = \|\text{sign}\,\Delta W L\|_F = \sqrt{d_{in} \cdot d_{out}}$

$\therefore a \leq \frac{1}{\sqrt{d_{in}}}$

3(e): $\|\Delta W\|_2 = a \cdot \|U\|_2 \cdot \|V^T\|_2$

$= a \cdot 1 \cdot 1$

$= a$

$\therefore a = \sqrt{d_{out}}$

3(g) $\nabla_{X_n} L = C_n \cdot W_n^T \cdot \nabla_{X_{n+1}} L$

$\|C_n \cdot W_n^T\|_2 = C_n \cdot \sqrt{d_{out}} = 1$

$\therefore C_n = \frac{1}{\sqrt{d_{out}}}$

4. (a): $\begin{bmatrix} -40 & -40 & -40 \\ -40 & -40 & -40 \\ -40 & -40 & -40 \end{bmatrix}$

(b): (i): $\begin{bmatrix} -19 & -28 & -32 & -36 & -29 \\ -30 & -40 & -40 & -40 & -30 \\ -30 & -40 & -40 & -40 & -30 \\ -30 & -40 & -40 & -40 & -30 \\ 49 & 68 & 72 & 76 & 59 \end{bmatrix}$

(ii): $\begin{bmatrix} -19 & -32 & -29 \\ -30 & -40 & -30 \\ 49 & 72 & 59 \end{bmatrix}$

5. (a): weights: $C \cdot K^2 \cdot F$, biases: F

Shape: $\left(\left\lfloor \frac{W+2P-K}{S} \right\rfloor\right) \times \left(\left\lfloor \frac{H+2P-K}{S} \right\rfloor\right) \times F$

(b): $\left(\left\lfloor \frac{W-K}{S} \right\rfloor\right) \times \left(\left\lfloor \frac{H-K}{S} \right\rfloor\right) \times C$

(c): $L=1$, $Size = K$

$L=2$, $Size = 2K-1$

$L=3$, $Size = 3K-2$

$\vdots$

$Size = L(k-1)+1$

(d): Let R be the receptive field size before pooling, and let J be the distance between the receptives of two adjacent feature points in the original input.

$\because$ The size of the kernel is 2

$\therefore R_{after} = R + J$

Assume that the size of the item is $M \times N \times C$ before doing the downsampling, after doing the downsampling, the size will turn to $\frac{M}{2} \times \frac{N}{2} \times 2C$.

Therefore, the total amount of the feature values is reduced by half.

(e):

Layer	Number of Parameters	Dimension of Activations
Input	0	$28 \times 28 \times 1$
CONV3-10	100	$28 \times 28 \times 10$
POOL2	0	$14 \times 14 \times 10$
CONV3-10	$3 \times 3 \times 10 \times 10 + 10$	$14 \times 14 \times 10$
POOL2	0	$7 \times 7 \times 10$
FLATTEN	0	490
FC-3	$490 \times 3 + 3$	3

(f): $g_3 = g_2 = \begin{bmatrix} 0 \\ 0.8 \\ 0 \end{bmatrix}$

$g_4 = g_1 = \begin{bmatrix} 0.8 \\ 0 \\ 0 \end{bmatrix}$

7. (a): $H_{out} = \lfloor \frac{H_{in}+2P-K}{S} \rfloor + 1$, $W_{out} = \lfloor \frac{W_{in}+2P-K}{S} \rfloor + 1$

(i): $H_{out} = \lfloor \frac{32+2-3}{1} \rfloor + 1 = 32$, $W_{out} = \lfloor \frac{32+2-3}{1} \rfloor + 1 = 32$, $C_{out} = 64$

∴ Shape will be $[10, 64, 32, 32]$

(ii): $H_{out} = \lfloor \frac{32+0-4}{2} \rfloor + 1 = 15$, $W_{out} = \lfloor \frac{32+0-4}{1} \rfloor + 1 = 15$, $C_{out} = 64$

∴ Shape will be $[10, 64, 32, 32]$

(b): $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$ (c): $\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$

(d): (i): $K' = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$

$M * K' = \begin{bmatrix} 1+3+7+9 \end{bmatrix} = \begin{bmatrix} 20 \end{bmatrix}$

(ii): $r = 3 + (5-1) \times 1 = 7$

∴ 7×7 is the receptive field

Hand-Designing Filters

Convolutional layer, which is the most important building block of CNN, actively utilizes the concept of filters used in traditional image processing. Therefore, it is quite important to know and understand the types and operation of image filters. In this notebook, we will design convolution filters by hand to understand the operation of convolution.

```
# As usual, a bit of setup

import time
import numpy as np
import matplotlib.pyplot as plt
import requests
import random
from PIL import Image
from scipy import ndimage


seed = 7
random.seed(seed)
np.random.seed(seed)

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

imagenet_mean = np.array([0.485, 0.456, 0.406])
imagenet_std = np.array([0.229, 0.224, 0.225])

def show_image(image, title=''):
    # image is a grayscale [H, W] or RGB [H, W, 3] NumPy array.
    plt.imshow(np.clip(np.asarray(image) * 255, 0, 255).astype(np.uint8))
    plt.title(title, fontsize=16)
    plt.axis('off')
    return

def show_multiple_images(images=[], titles=[]):
    assert len(images) == len(titles), "length of two inputs are not equal"
    N = len(images)
    # make the plt figure larger
    plt.rcParams['figure.figsize'] = [24, 24]

    for i in range(N):
        plt.subplot(1, N, i+1)
        show_image(images[i], titles[i])

    plt.show()

def rgb2gray(rgb):
    r, g, b = rgb[:,:,0], rgb[:,:,1], rgb[:,:,2]
    gray = 0.2989 * r + 0.5870 * g + 0.1140 * b

    return gray
```

Designing Filters

In this problem, you will design simple blurring and edge detection filters.

```
img_url = 'https://user-images.githubusercontent.com/11435359/147738734-196fd92f-9260-48d5-ba7e-bf103d29364d.jpg' # fox, from ILSVRC2012_val_00046145

img = Image.open(requests.get(img_url, stream=True).raw)
img = np.array(img) / 255
gray_img = rgb2gray(img)


show_image(gray_img, 'Original Image')
```

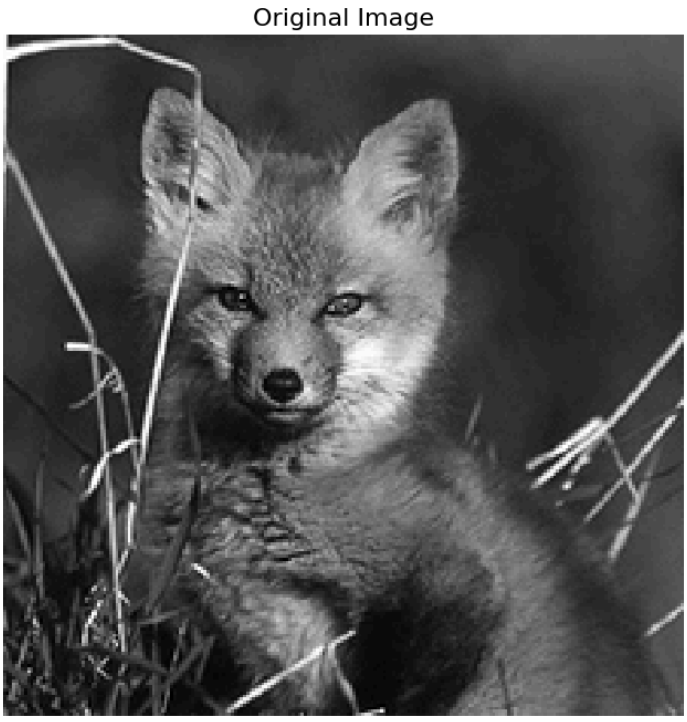


Image Blurring

Image blurring also called image smoothing, usually refers to making an image fuzzy. This filtering is typically used to remove noise in the image. There are various types of image blurring filters, but the three most common are Averaging, Gaussian blurring, and Median filtering.

We will implement Averaging filtering in this project. Averaging filtering is also called moving averaging in 1-D. This filter works by placing a mask over an image and then taking the average of all the image pixels covered by the mask and replacing the central pixel with that value.

If the kernel size of the image filter is $n \times n$, then the size of each element in the kernel matrix is $\frac{1}{n^2}$. Also, the sum of all the elements in the kernel matrix will be 1. So, if the kernel size is 3×3, kernel will be as follows.

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

```
def averaging_filtering(image, filter_size=3):
        ###############################################################################
        # TODO: Implement the averaging filter with the given filter size.                    #
        # Hint: You can use np.ones                                                
```

Canny edge detector, the Sobel filter and the Laplacian derivatives filter.

Here, we will use the 3×3 Laplacian kernel. This kernel, when convolved with the image, computes the 3×3 discrete Laplacian of the image:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

```
def edge_detecting(image):
    ###############################################################################
    # TODO: Implement the Laplacian derivative filter.                            #
    ###############################################################################
    kernel = np.array(([0,1,0],[1,-4,1],[0,1,0]))
    ###############################################################################
    #                                 END OF YOUR CODE                            #
    ###############################################################################
    output = ndimage.convolve(image, kernel)
    return output

edge_images, edge_titles = [gray_img], ['original']
edge_image = edge_detecting(gray_img)
edge_images.append(edge_image)
edge_titles.append(f'Edge Detection')

show_multiple_images(edge_images, edge_titles)
```

